

21世纪软件工程专业规划教材

软件工程导论（第6版）

第8章 面向对象分析

面向对象分析（OOA）的关键是识别出问题域内的类与对象，并分析它们相互间的关系，最终建立起问题域的简洁、精确、可理解的正确模型。在用面向对象观点建立起的3种模型中，对象模型是最基本、最重要、最核心的。

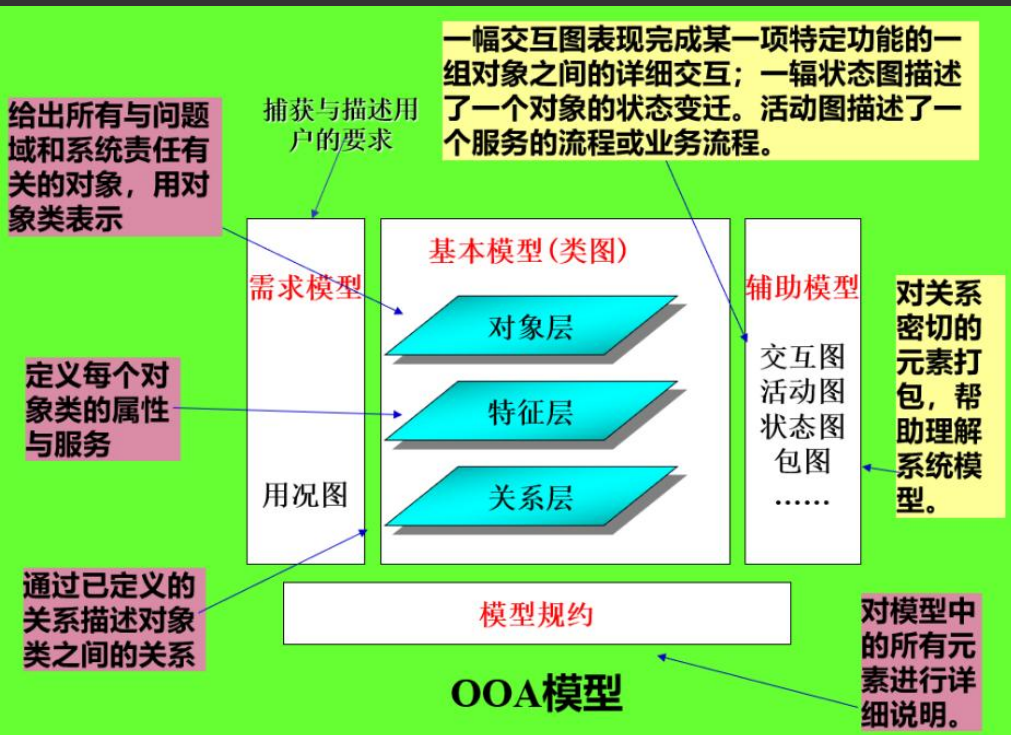
目录

CONTENTS

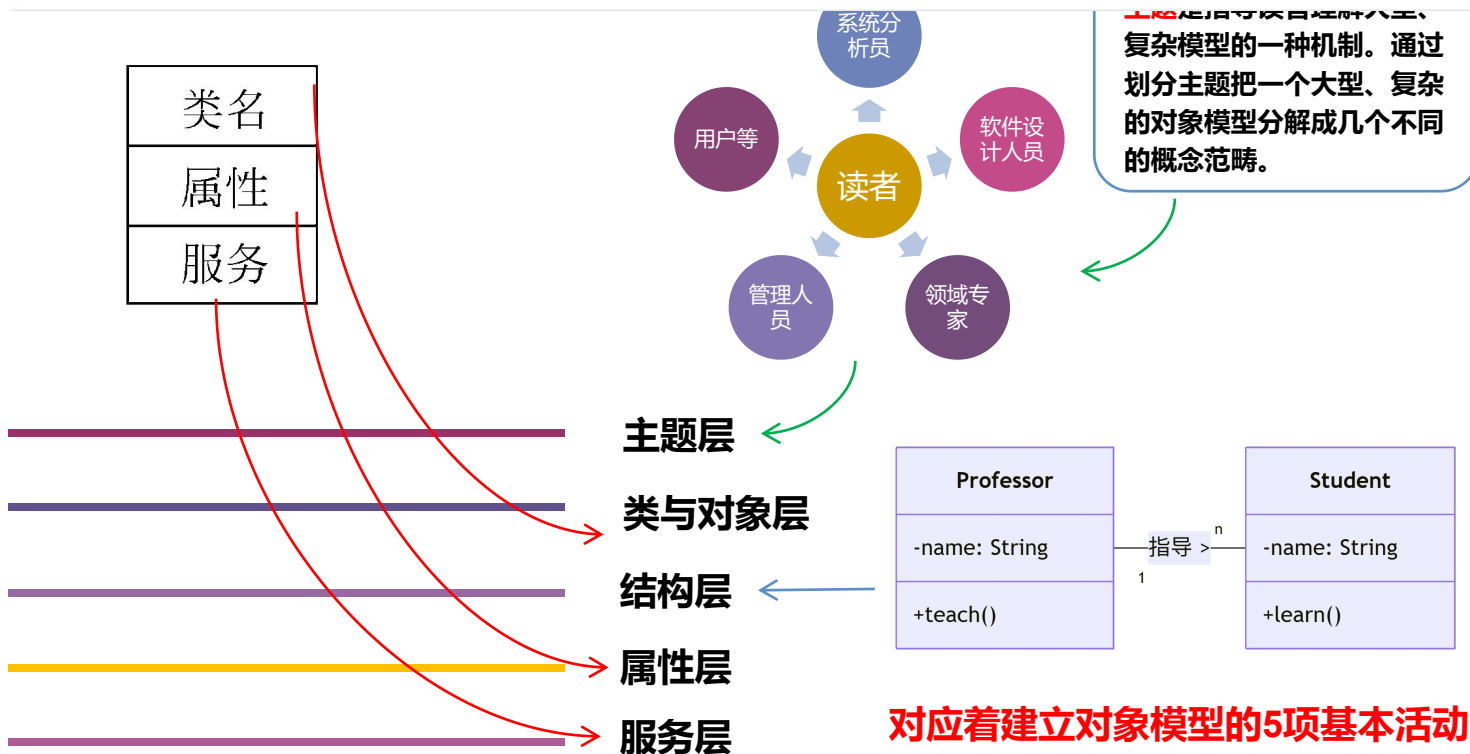
- 1 分析的基本过程
- 2 需求陈述
- 3 建立对象模型
- 4 建立动态模型
- 5 建立功能模型
- 6 定义服务

分析的基本过程

面向对象分析：就是抽取和整理用户需求，将问题域中的事物抽象为系统中的对象，识别对象的特征和各类对象之间的关系，建立一个映射问题域并能满足用户需求的面向对象分析模型，即OOA模型。

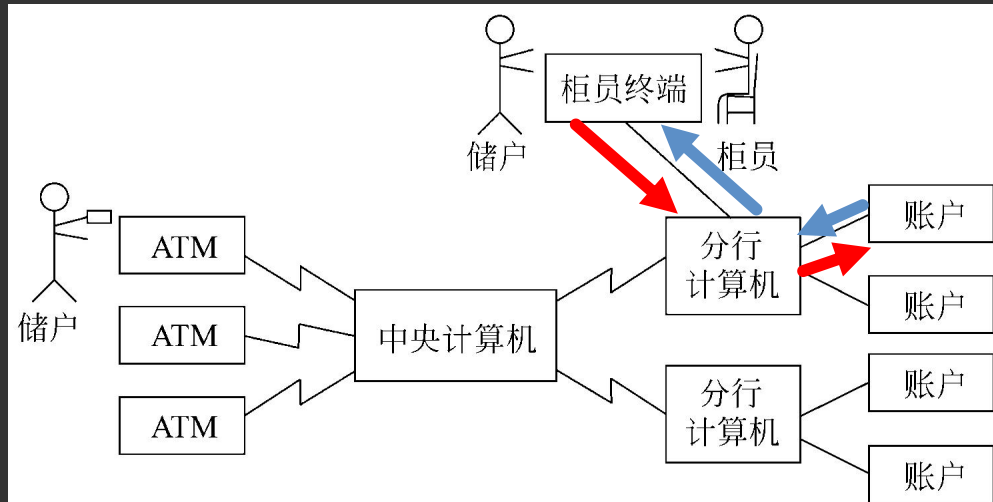


层次	具体内容
主题层	1. 图书编目主题 2. 借阅流通主题 3. 会员管理主题
类与对象层 (在借阅流通主题内)	BookCopy , Loan , Member , Reservation , Fine
结构层	Member → Loan (1对多) ; BookCopy → Loan (1对1) ; Loan → Fine (0..1对1)
属性层	Loan : loanId , borrowDate , dueDate , returnDate , status
服务层	Loan 类的方法: checkout() , returnBook() , renew() , calculateFine()



需求陈述

需求陈述应该阐明“做什么”而不是“怎样做”！

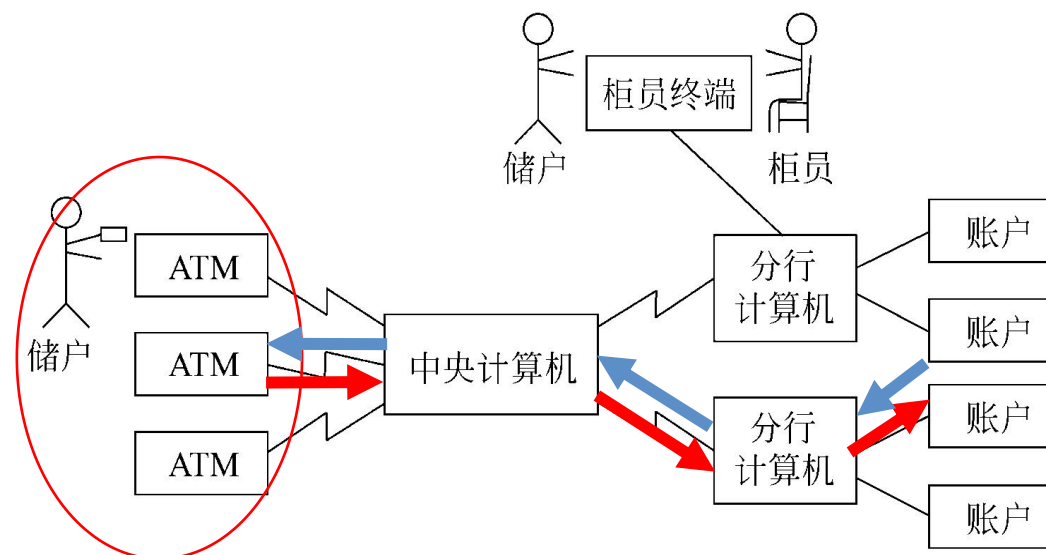


某银行拟开发一个自动取款机系统，它是一个由自动取款机、中央计算机、分行计算机及柜员终端组成的网络系统。

ATM和中央计算机由总行投资购买。总行拥有多台ATM，分别设在全市各主要街道上。分行负责提供分行计算机和柜员终端。柜员终端设在分行营业厅及分行下属的各个储蓄所内。该系统的软件开发成本由各个分行分摊。

② 储户和ATM交互

- ATM要求用户输入密码。
- ATM把从这张卡上读到的信息以及用户输入的密码传给中央计算机，请求中央计算机核对这些信息并处理这次事务
- 中央计算机根据卡上的分行代码确定这次事务与分行的对应关系，并且委托相应的分行计算机验证用户密码。
- 用户输入的密码是正确的，ATM就要求用户选择事务类型（取款、查询等）。当用户选择取款时，ATM请求用户输入取款额。
- ATM从现金出口吐出现金，并且打印出账单交给用户。



建立对象模型

1. 确定类与对象

- ① 找出候选的类与对象：**非正式分析**（用自然语言书写的需求陈述为依据，把**名词**作为类与对象的候选者；用**形容词**作为确定属性的线索；把**动词**作为服务（操作）的候选者。）

阅读10.2.2节给出的需求陈述，从陈述中找出下列名词，可以把它们作为类与对象的初步的候选者：

银行，自动取款机ATM，系统，中央计算机，分行计算机，柜员终端，网络，总行，分行，软件，成本，市，街道，营业厅，储蓄所，柜员，储户，现金，支票，账户，事务，现金兑换卡，余额，磁卡，分行代码，卡号，用户，副本，信息，密码，类型，取款额，账单，访问。

通常，在需求陈述中不会一个不漏地写出问题域中所有有关的类与对象，因此，分析员应该根据领域知识或常识进一步把**隐含的类与对象**提取出来。例如，在ATM系统的需求陈述中虽然没写“通信链路”和“事务日志”，但是，根据领域知识和常识可以知道，在ATM系统中应该包含这两个实体。

- ② 筛选出正确的类与对象：**严格考察每个候选对象，从中去掉不正确的或不必要的，仅保留确实应该记录其信息或需要其提供服务的那些对象。**

- 冗余：两个类表达了同样的信息，则应该保留在此问题域中最富于描述力的名称。

ATM系统的储户与用户，现金兑换卡与磁卡及副本分别描述了相同的两类信息，因此，应该去掉“**用户**”、“**磁卡**”、“**副本**”等冗余的类，仅保留“储户”和“现金兑换卡”这两个类。

银行，自动取款机ATM，系统，中央计算机，分行计算机，柜员终端，网络，总行，分行，软件，成本，市，街道，营业厅，储蓄所，柜员，储户，现金，支票，账户，事务，现金兑换卡，余额，**磁卡**，分行代码，卡号，**用户**，**副本**，信息，密码，类型，取款额，账单，访问。

建立对象模型

- 无关：仅需要把**与本问题密切相关**的类与对象放进目标系统中。有些类在其他问题中可能很重要，但与当前要解决的问题无关，同样也应该把它们删掉。

以ATM系统为例，这个系统并不处理分摊软件开发成本的问题，而且ATM和柜员终端放置的地点与本软件的关系也不大。因此，应该去掉候选类“**成本**”、“**市**”、“**街道**”、“**营业厅**”和“**储蓄所**”。

银行，自动取款机ATM，系统，中央计算机，分行计算机，柜员终端，网络，总行，分行，软件，**成本**，**市**，**街道**，**营业厅**，**储蓄所**，柜员，储户，现金，支票，账户，事务，现金兑换卡，余额，**磁卡**，分行代码，卡号，**用户**，**副本**，信息，密码，类型，取款额，账单，访问。

- 笼统：在需求陈述中常常使用一些笼统的、泛指的名词，要么系统无须记忆有关它们的信息，要么在需求陈述中有更明确更具体的名词对应它们所暗示的事务，因此，通常把这些笼统的或模糊的类去掉。

以ATM系统为例，“银行”实际指总行或分行，“访问”在这里实际指事务，“信息”的具体内容在需求陈述中随后就指明了。总之，在本例中应该去掉“**银行**”、“**网络**”、“**系统**”、“**软件**”、“**信息**”、“**访问**”等候选类。

银行，自动取款机ATM，**系统**，中央计算机，分行计算机，柜员终端，**网络**，总行，分行，**软件**，**成本**，**市**，**街道**，**营业厅**，**储蓄所**，柜员，储户，现金，支票，账户，事务，现金兑换卡，余额，**磁卡**，分行代码，卡号，**用户**，**副本**，**信息**，密码，类型，取款额，账单，**访问**。

建立对象模型

- 属性: 在需求陈述中有些名词实际上描述的是其他对象的属性，应该把这些名词从候选类与对象中去掉。当然，如果某个性质具有很强的独立性，则应把它作为类而不是作为属性。

以ATM系统为例，“现金”、“支票”、“取款额”、“账单”、“余额”、“分行代码”、“卡号”、“密码”、“类型”等，都应该作为属性对待。

银行, 自动取款机ATM, 系统, 中央计算机, 分行计算机, 柜员终端, 网络, 总行, 分行, 软件, 成本, 市, 街道, 营业厅, 储蓄所, 柜员, 储户, 现金, 支票, 账户, 事务, 现金兑换卡, 余额, 磁卡, 分行代码, 卡号, 用户, 副本, 信息, 密码, 类型, 取款额, 账单, 访问。

- 实现: 在分析阶段不应该过早地考虑怎样实现目标系统，应该去掉仅和实现有关的候选的类与对象。在设计和实现阶段，这些类与对象可能是重要的，但在分析阶段过早地考虑它们反而会分散人们的注意力。

在ATM系统中，“事务日志”无非是对一系列事务的记录，它的确切表示方式是面向对象设计的议题；“通信链路”在逻辑上是一种联系，在系统实现时它是关联类的物理实现。应该暂时去掉“事务日志”和“通信链路”这两个类，在设计或实现时再考虑它们。

银行, 自动取款机ATM, 系统, 中央计算机, 分行计算机, 柜员终端, 网络, 总行, 分行, 软件, 成本, 市, 街道, 营业厅, 储蓄所, 柜员, 储户, 现金, 支票, 账户, 事务, 现金兑换卡, 余额, 磁卡, 分行代码, 卡号, 用户, 副本, 信息, 密码, 类型, 取款额, 账单, 访问。

- 操作: 在需求陈述中有时可能使用一些既可作为名词，又可作为动词的词，应该慎重考虑它们在本问题中的含义，以便正确地决定把它们作为类还是作为类中定义的操作。

建立对象模型

2. 确定关联

两个或多个对象之间的相互依赖、相互作用关系就是关联。分析确定关联，能使分析员考虑问题域的边缘情况，有助于发现那些尚未被发现的类与对象。

① 初步确定关联

在需求陈述中使用的描述性动词或动词词组，通常表示关联关系。

- 直接提取动词短语得出的关联
- ATM、中央计算机、分行计算机及柜员终端组成网络。
 - 总行拥有多台ATM。
 - ATM设在主要街道上。
 - 分行提供分行计算机和柜员终端。
 - 柜员终端设在分行营业厅及储蓄所内。
 - 分行分摊软件开发成本。
 - 储户拥有账户。
 - 分行计算机处理针对账户的事务。
 - 分行计算机维护账户。
 - 柜员终端与分行计算机通信。
 - 柜员输入针对账户的事务。
 - ATM与中央计算机交换关于事务的信息。
 - 中央计算机确定事务与分行的对应关系。
 - ATM读现金兑换卡。
 - ATM与用户交互。
 - ATM吐出现金。
 - ATM打印账单。
 - 系统处理并发的访问。

- 需求陈述中隐含的关联
- 总行由各个分行组成。
 - 分行保管账户。
 - 总行拥有中央计算机。
 - 系统维护事务日志。
 - 系统提供必要的安全性。
 - 储户拥有现金兑换卡。
- 根据问题域知识得出的关联
- 现金兑换卡访问账户。
 - 分行雇用柜员。

建立对象模型

② 筛选

已删去的类之间的关联

如果在分析确定类与对象的过程中已经删掉了某个候选类，则与这个类有关的关联也应该删去，或用其他类重新表达这个关联。

以ATM系统为例，由于已经删去了“系统”、“网络”、“市”、“街道”、“成本”、“软件”、“事务日志”、“现金”、“营业厅”、“储蓄所”、“账单”等候选类，因此与这些类有关的下列8个关联也应该删去。	- ATM、中央计算机、分行计算机及柜员终端组成网络。 - 总行拥有多台ATM。 - ATM设在主要街道上。 - 分行提供分行计算机和柜员终端。 - 柜员终端设在分行营业厅及储蓄所内。 - 分行分摊软件开发成本。 - 储户拥有账户。 - 分行计算机处理针对账户的事务。
应该把处在 本问题域之外 的关联或与 实现 密切相关的关联删去。	在ATM系统的例子中，“系统处理并发的访问”并没有标明对象之间的新关联，它只不过提醒人们在实现阶段需要使用实现并发访问的算法，以处理并发事务。
关联应该描述问题域的 静态结构 ，而不应该是一个瞬时事件	以ATM系统为例，“ATM读现金兑换卡”描述了ATM与用户交互周期中的一个动作，它并不是ATM与现金兑换卡之间的固有关系，因此应该删去。类似地，还应该删去“ATM与用户交互”这个候选的关联。
三个或三个以上对象之间的关联，大多可以分解为 二元关联 或用词组描述成限定的关联。	“柜员输入针对账户的事务”可以分解成“柜员输入事务”和“事务修改账户”这样两个二元关联。而且“分行计算机处理针对账户的事务”也可以做类似的分解。 “ATM与中央计算机交换关于事务的信息”这个候选的关联，实际上隐含了“ATM与中央计算机通信”和“在ATM上输入事务”这两个二元关联。

建立对象模型



3. 划分主题

在开发大型、复杂系统的过程中，为了降低复杂程度，人们习惯于把系统再进一步划分成几个不同的主题，也就是在概念上把系统包含的内容分解成若干个范畴。应该按问题领域而不是用功能分解方法来确定主题。此外，应该按照使不同主题内的对象相互间依赖和交互最少的原则来确定主题。

以ATM系统为例，可以把它划分成3个主题：

总行（包含总行和中央计算机这两个类）

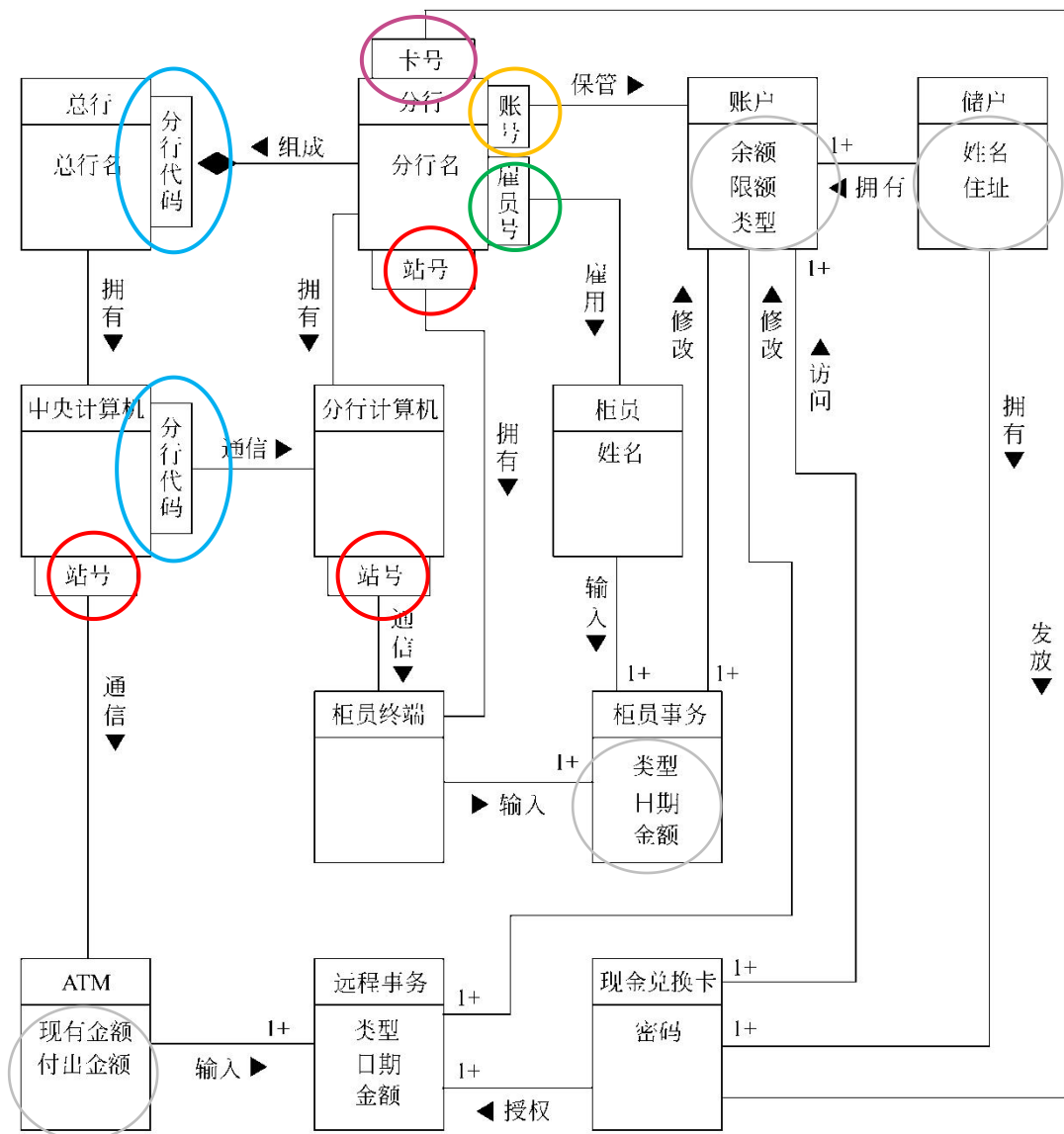
分行（包含分行、分行计算机、柜员终端、柜员事务、柜员和账户等类）

ATM（包含ATM、远程事务、现金兑换卡和储户等类）

4. 确定属性

一般说来，确定属性的过程包括分析和选择两个步骤。

- ① 分析：通常需求陈述中用名词词组表示属性，例如，“账户的余额”或“远程事务的类型”等。另外可以用形容词表示可枚举的具体属性，例如，订单类的属性“订单状态”可以枚举的值可以是“已支付、未支付或已取消”。
- ② 选择：认真考察经初步分析而确定下来的那些属性，从中删掉不正确的或不必要的属性。通常有以下几种常见情况：
 - 误把对象当作属性
 - 误把内部状态当成了属性
 - 误把关联类的属性当作一般对象的属性
 - 过于细化
 - 把限定误当成属性
 - 存在不一致的属性



“卡号” “分行发放现金兑换卡” 这个关联的限定词

“分行代码” 是关联 “分行组成总行” 上的限定词。

“账号” 是关联 “分行保管账户” 上的限定词。

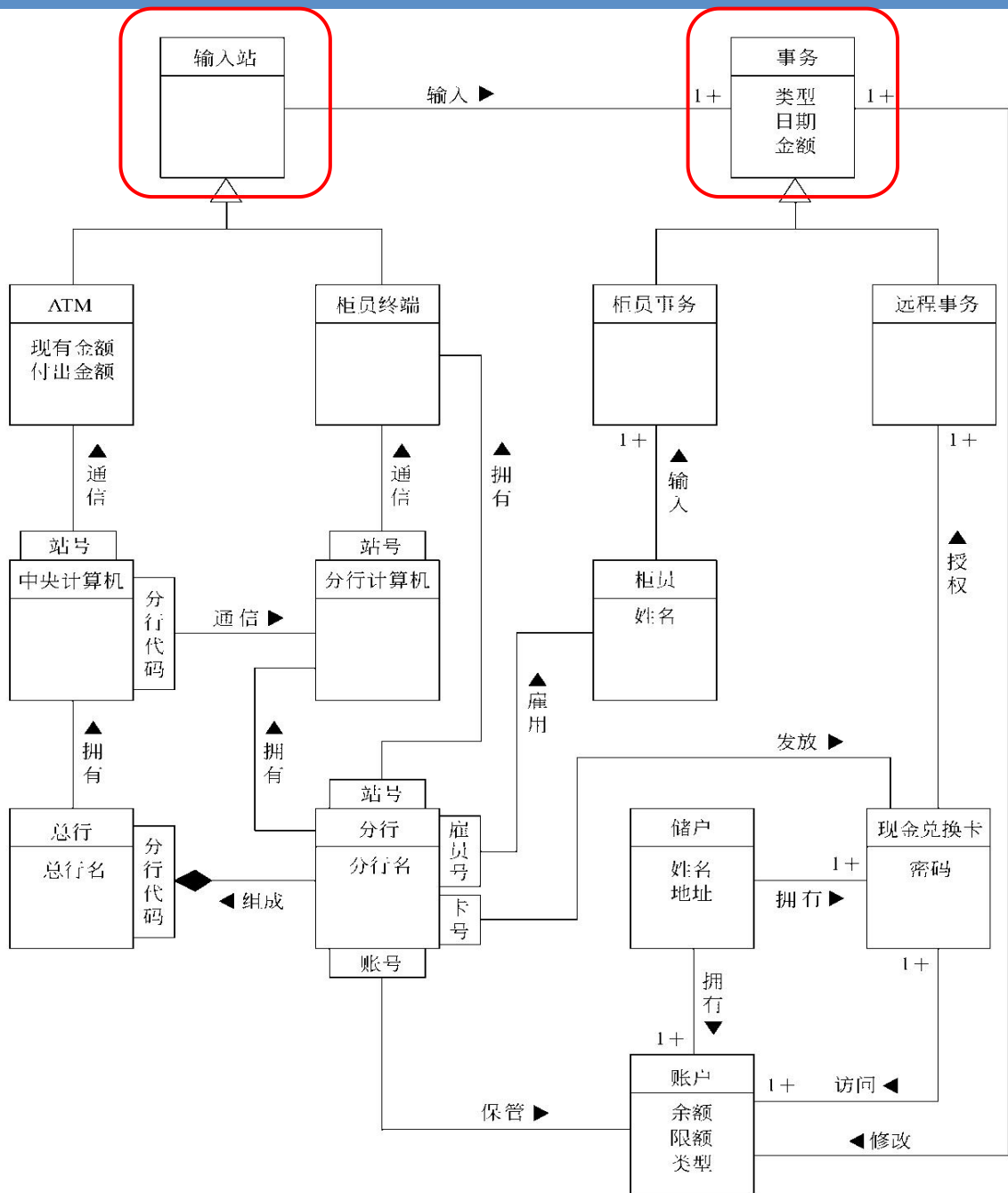
“雇员号” 是 “分行雇用柜员” 上的限定词。

“站号” 是 “分行拥有柜员终端”、“柜员终端与分行计算机通信” 及 “中央计算机与ATM通信” 3个关联上的限定词。

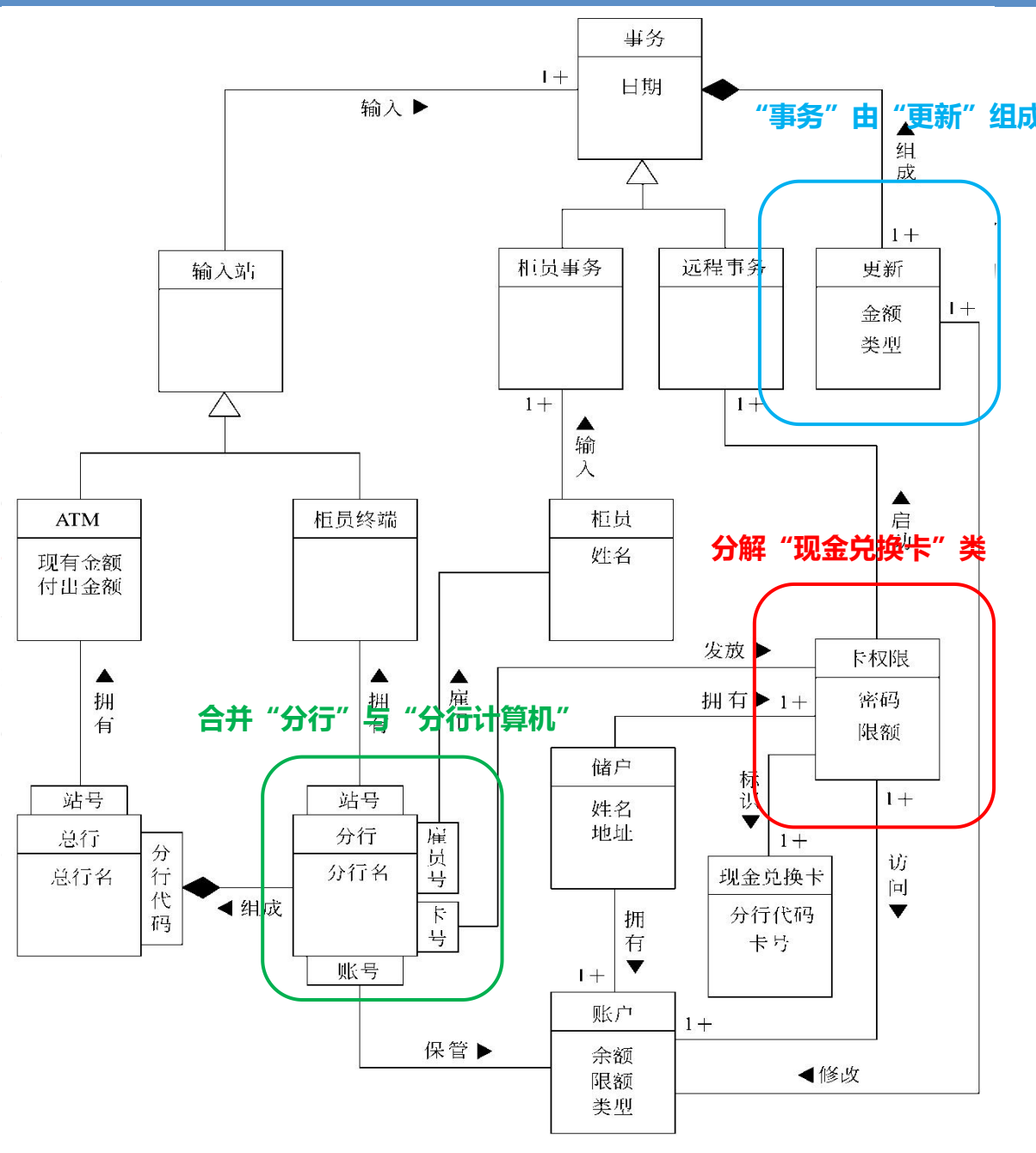
建立对象模型

5

面向对象

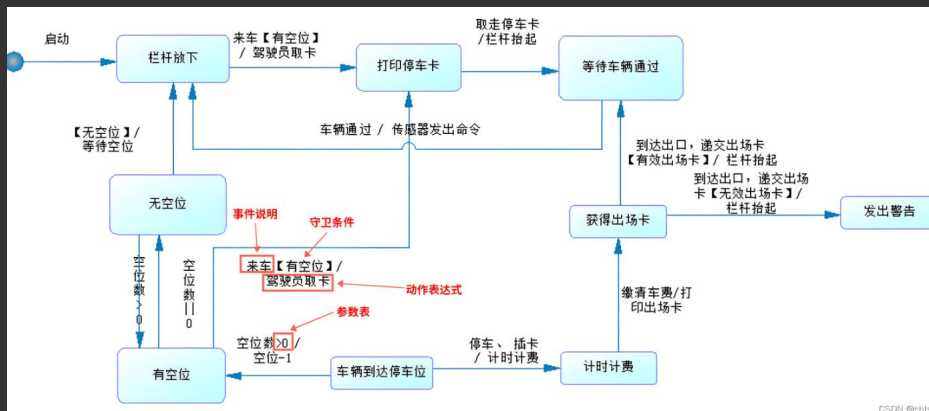


建立对象模型



建立动态模型

在开发**交互式系统**时，动态模型起着非常重要的作用。如果收集输入信息是目标系统的一项重要工作，则在开发此类应用系统时建立正确的动态模型至关重要。



① **编写典型交互行为的脚本**。虽然脚本中不可能包括每个偶然事件，但是，至少必须保证不遗漏常见的交互行为。

② **从脚本中提取出事件**，确定触发每个事件的动作对象以及接受事件的目标对象。

③ **排列事件发生的次序**，确定每个对象可能的状态及状态间的转换关系，并用状态图描绘它们。

④ **比较各个对象的状态图**，检查它们之间的一致性，确保事件之间的匹配。

在建立动态模型的过程中，**脚本**是指系统在**某一执行期间内出现的一系列事件**。脚本描述用户（或其他外部设备）与目标系统之间的一个或多个典型的**交互过程**（事件序列），以便对目标系统的行为有更具体的认识。**目的**保证**不遗漏重要的交互步骤**，有助于确保整个交互过程的正确性和清晰性。

正常情况的脚本

- ATM请储户插卡；储户插入一张现金兑换卡。
- ATM接受该卡并读它上面的分行代码和卡号。
- ATM要求储户输入密码；储户输入自己的密码“1234”等数字。
- ATM请求总行验证卡号和密码；总行要求“39”号分行核对储户密码，然后通知ATM说这张卡有效。
- ATM要求储户选择事务类型（取款、转账、查询等）；储户选择“取款”。
- ATM要求储户输入取款额；储户输入“880”。
- ATM确认取款额在预先规定的限额内，然后要求总行处理这个事务；总行把请求转给分行，该分行成功地处理完这项事务并返回该账户的新金额。
- ATM吐出现金并请储户拿走这些现金；储户拿走现金。
- ATM问储户是否继续这项事务；储户回答“不”。
- ATM打印账单，退出现金兑换卡，请储户拿走它们；储户取走账单和卡。
- ATM请储户插卡。

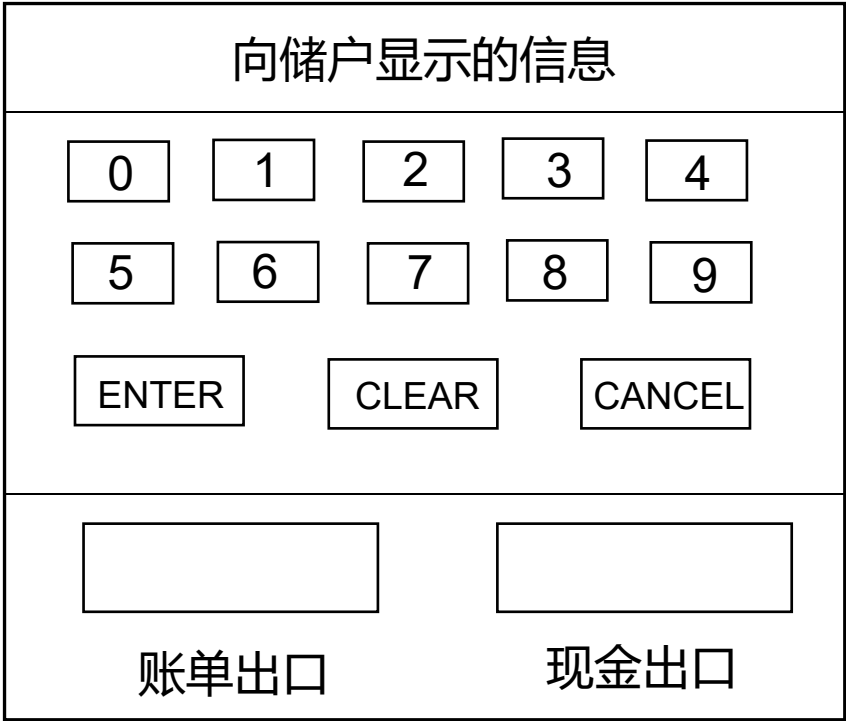
异常情况的脚本

- ATM请储户插卡，储户插入一张现金兑换卡。
- ATM接受该卡并顺序读它上面的数字。
- ATM要求密码:储户误输入“888等数字”。
- ATM请求总行验证卡号和密码:总行在向有关分行咨询后拒绝这张卡。
- ATM显示“密码错”，并请储户输入密码:储户输入“1234”等数字: ATM请求总行验证后知道输入密码正确。
- ATM要求储户选择事务类型:储户选择“取款”。
- ATM询问取款额:储户改变主意不想取款了，按“取消”。
- ATM退出现金兑换卡，请储户拿走它们:储户取走卡。
- ATM请储户插卡。

脚本描述**事件序列**，对于每个事件都应指明c
触发该事件的**动作对象**、接受事件的**目标对象**
以及改**事件的参数**。

大多数交互行为都可以分为**应用逻辑**和**用户界面**两部分。通常，系统分析员首先集中精力考虑系统的**信息流**和**控制流**，而不是首先考虑用户界面。

采用不同界面（如命令行或图形用户界面），可以实现同样的程序逻辑。应用逻辑是内在的、本质的内容，用户界面是外在的表现形式。**动态模型着重表示应用系统的控制逻辑。**



ATM的界面格式

① 确定事件

事件包括系统与用户（或外部设备）交互的所有**信号、输入、输出、中断、动作**等。从脚本中容易找出**正常事件**，但不要遗漏了**异常事件**和**出错条件**。

- 从每个**脚本**中提取出所有**外部事件**。
- 传递信息的对象**的动作也是事件。例如，储户插入现金兑换卡、储户输入密码、ATM吐出现金等都是事件。
- 大多数对象到对象的交互行为都对应着事件。

对控制流产生相同效果的事件组合在一起作为一类事件，并给它们取唯一的名字。

例如，“吐出现金”是一个事件类，尽管这类事件中的每个个别事件的参数值不同（吐出的现金数额不同），然而这并不影响控制流。

应把对控制流有不同影响的那些事件区分开来，不要误把它们组合在一起。

例如，“账户有效”、“账户无效”、“密码错”等都是不同的事件。

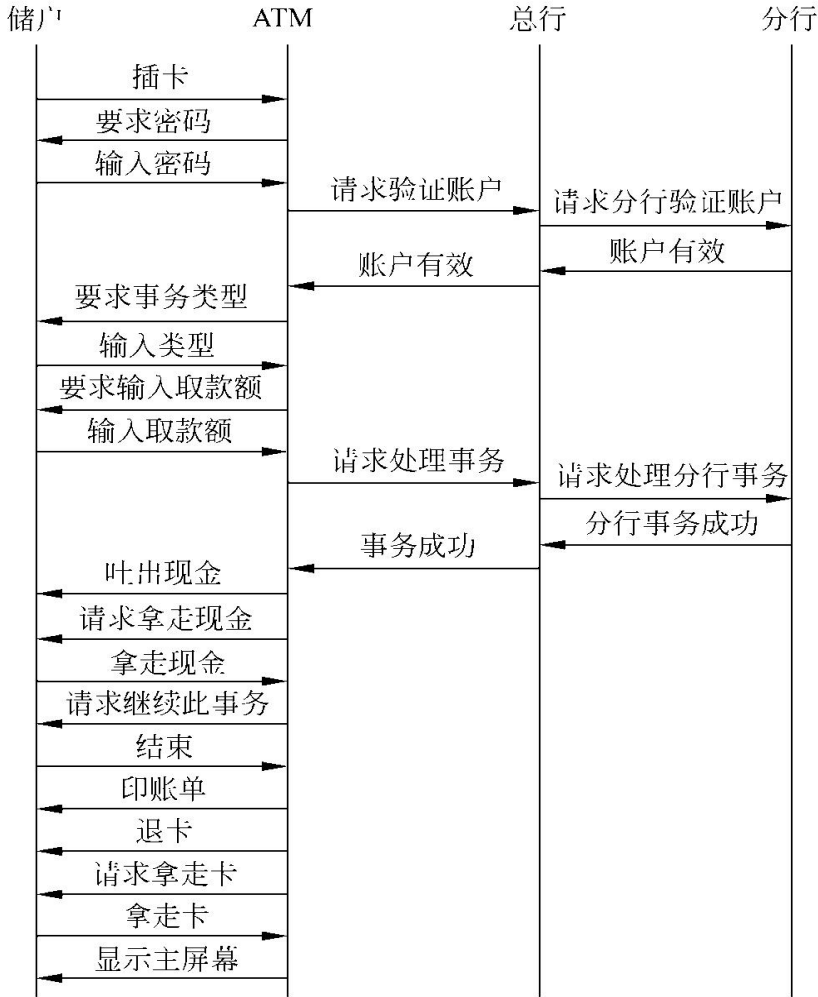
经过分析，应该区分出每类事件的**发送对象**和**接受对象**。一类事件相对它的发送对象来说是**输出事件**，但是相对它的接受对象来说则是**输入事件**。有时一个对象把事件发送给自己，在这种情况下，该事件既是输出事件又是输入事件。

画出事件跟踪图

② 绘制事件跟踪图

事件跟踪图实质上是扩充的脚本，可以认为事件跟踪图是简化的UML顺序图。

在事件跟踪图中，一条竖线代表一个对象，每个事件用一条水平的箭头线表示，箭头方向从事件的发送对象指向接受对象。时间从上向下递增，



画状态图

状态图描绘事件与对象状态的关系。当对象接受了一个事件以后，它的下个状态取决于当前状态及所接受的事件。由事件引起的状态改变称为“转换”。通常用一张状态图描绘一类对象的行为，它确定了由事件序列引出的状态序列。我们需要且仅考虑事件跟踪图中指向某条竖线的那些箭头线。

① 考虑正常事件

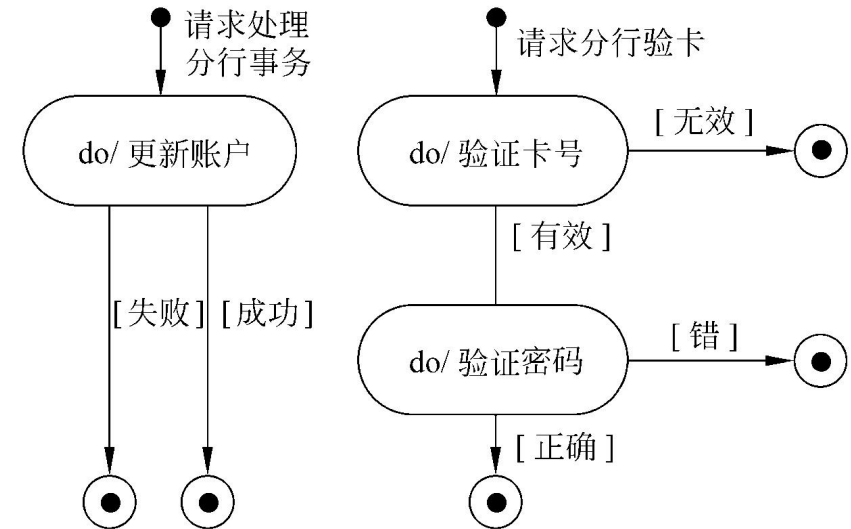
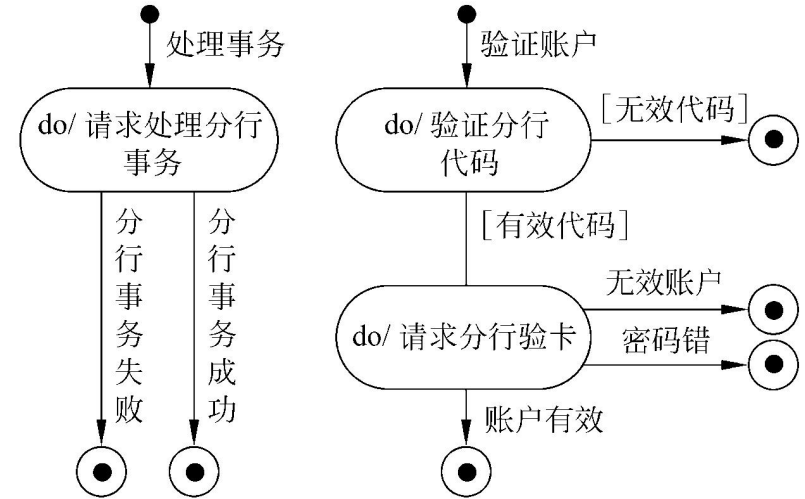
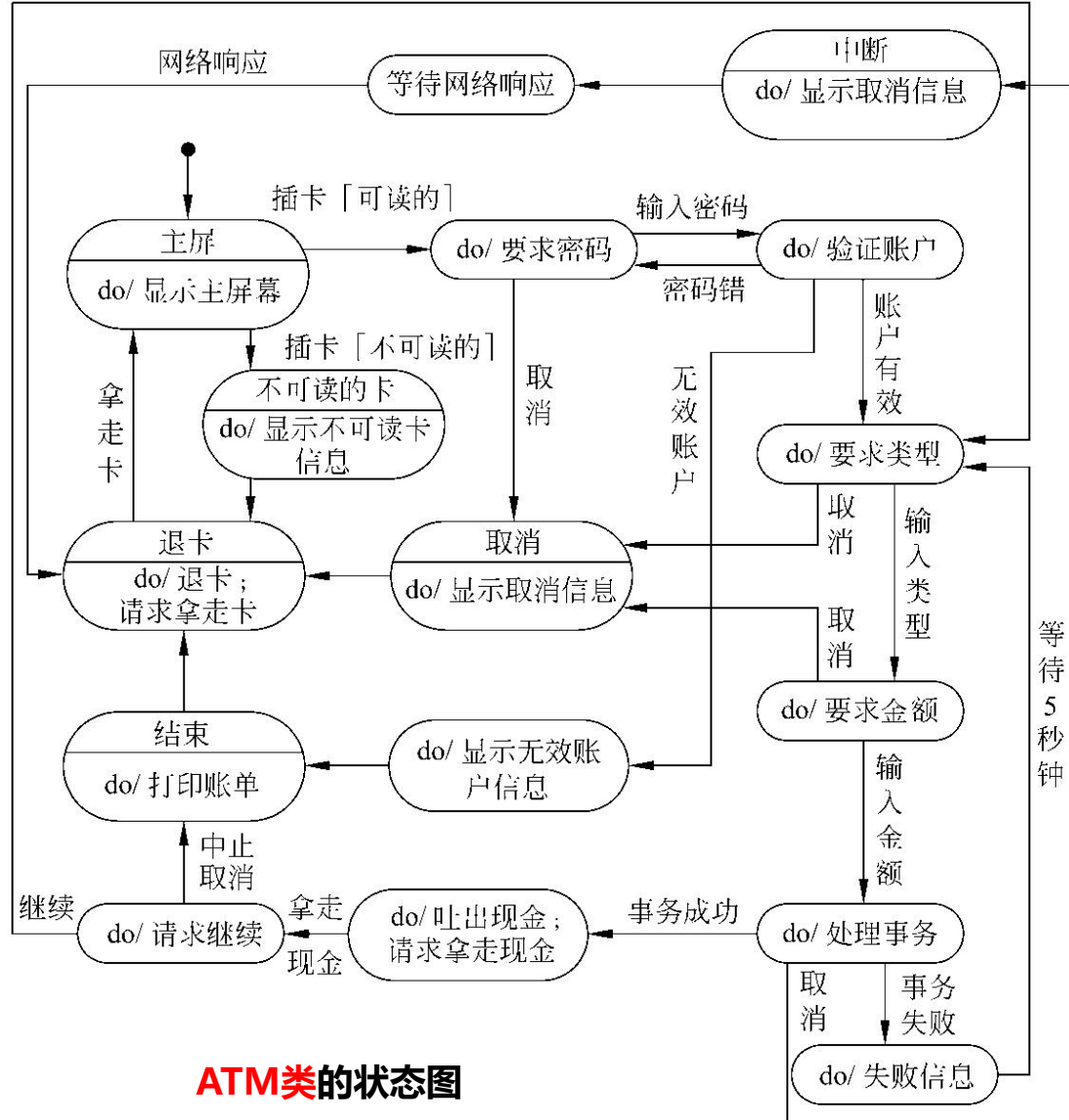
根据一张事件跟踪图画出状态图之后，再把其他脚本的事件跟踪图合并到已画出的状态图中。为此需在事件跟踪图中找出以前考虑过的脚本的分支点（例如“验证账户”就是一个分支点，因为验证的结果可能是“账户有效”，也可能是“无效账户”），然后把其他脚本中的事件序列并入已有的状态图中，作为一条可选的路径。

② 考虑完正常事件之后再考虑边界情况和特殊情况

其中包括在不适当时候发生的事件（例如系统正在处理某个事务时，用户要求取消该事务）。有时用户（或外部设备）不能做出快速响应，然而某些资源又必须及时收回，于是在一定间隔后就产生了“超时”事件。对用户出错情况往往需要花费很多精力处理，并且会使原来清晰、紧凑的程序结构变得复杂、繁琐，但出错处理是不能省略的。

“ATM”、“柜员终端”、“总行”和“分行”都是主动对象，它们相互发送事件；而“现金兑换卡”、“事务”和“账户”是被动对象，并不发送事件。“储户”和“柜员”虽然也是动作对象，但是，它们都是系统外部的因素，无须在系统内实现它们。因此，只需要考虑“ATM”、“总行”、“柜员终端”和“分行”的状态图。

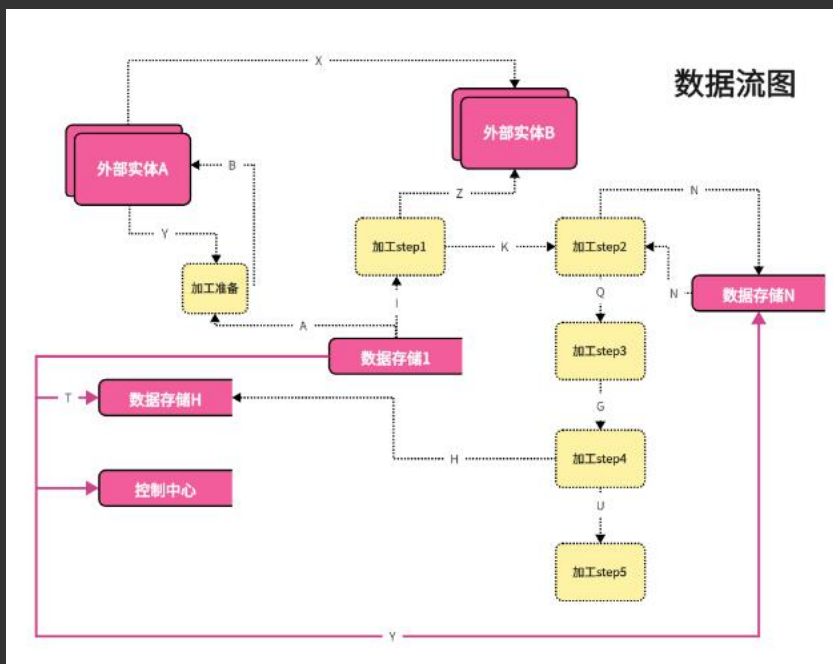
画状态图



建立功能模型

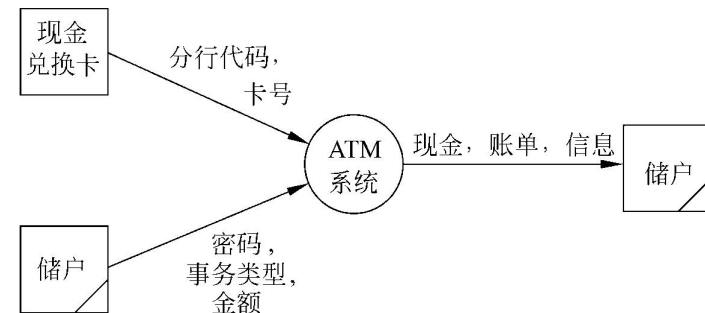
功能模型表明了系统中数据之间的依赖关系，以及有关的数据处理功能，它由一组**数据流图**组成。其中的处理功能可以用IPO图（或表）、伪码等多种方式进一步描述。

通常在建立了对象模型和动态模型之后再建立功能模型。



基本系统模型由若干个**数据源点/终点**，及一个**处理框**组成，这个处理框代表了系统加工、变换数据的整体功能。基本系统模型指明了**目标系统的边界**。由数据源点输入的数据和输出到数据终点的数据，是系统与外部世界之间的交互事件的参数。

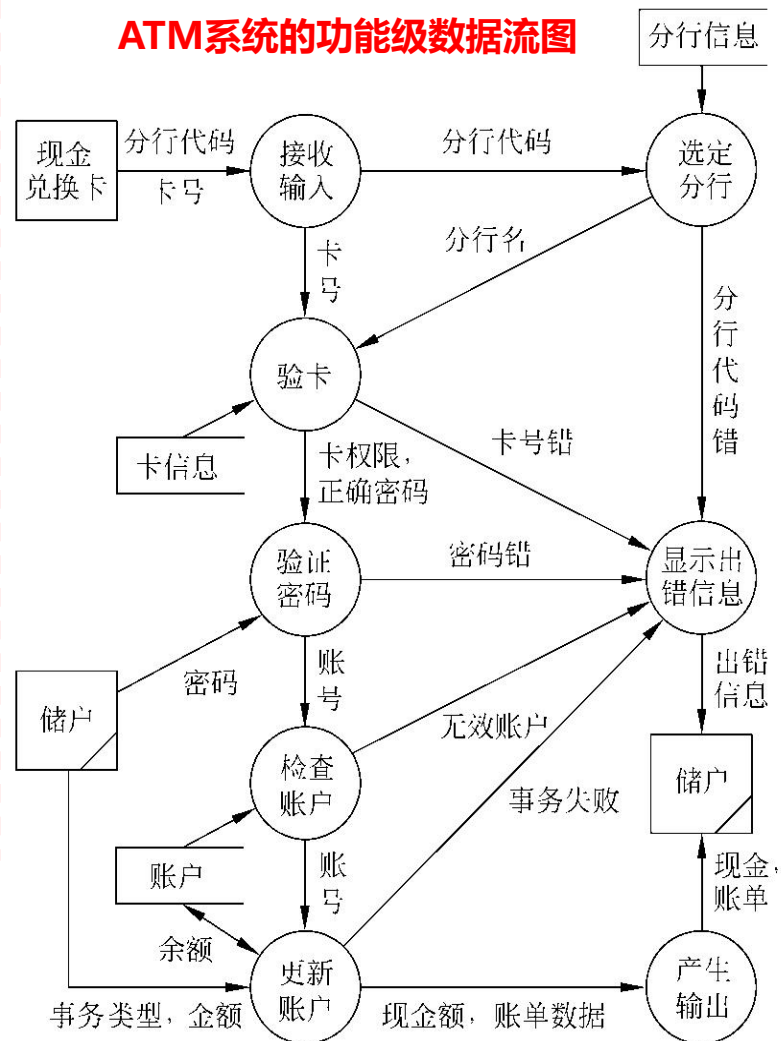
下图是ATM系统的基本系统模型。尽管在储蓄所内储户的事务是由柜员通过柜员终端提交给系统的，但是信息的来源和最终接受者都是储户，因此，本系统的数据源点/终点为储户。另一个数据源点是现金兑换卡，因为系统从它上面读取分行代码和卡号等信息。



把基本系统模型中单一的处理框分解成若干个处理框，以描述系统加工、变换数据的基本功能，就得到功能级数据流图。



ATM系统的功能级数据流图

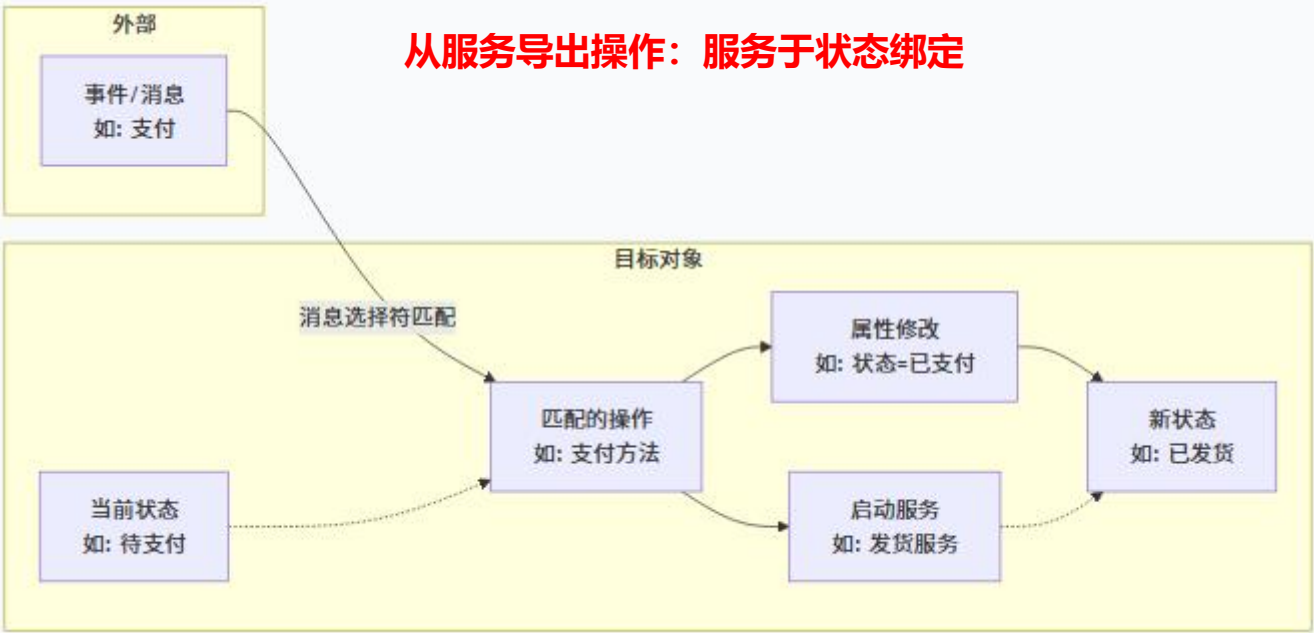


定义服务

“对象”是由描述其属性的数据，及可以对这些数据施加的操作（即服务），封装在一起构成的独立单元。因此，为建立完整的对象模型，既要确定类中应该定义的属性，又要确定类中应该定义的服务。



需要等到建立了**动态模型**和**功能模型**之后，才能最终确定类中应有的**服务**，因为这两个子模型更明确地描述了每个类中应该提供哪些服务。事实上，在确定类中应有的服务时，既要考虑该类实体的**常规行为**，又要考虑在本系统中**特殊需要的服务**。



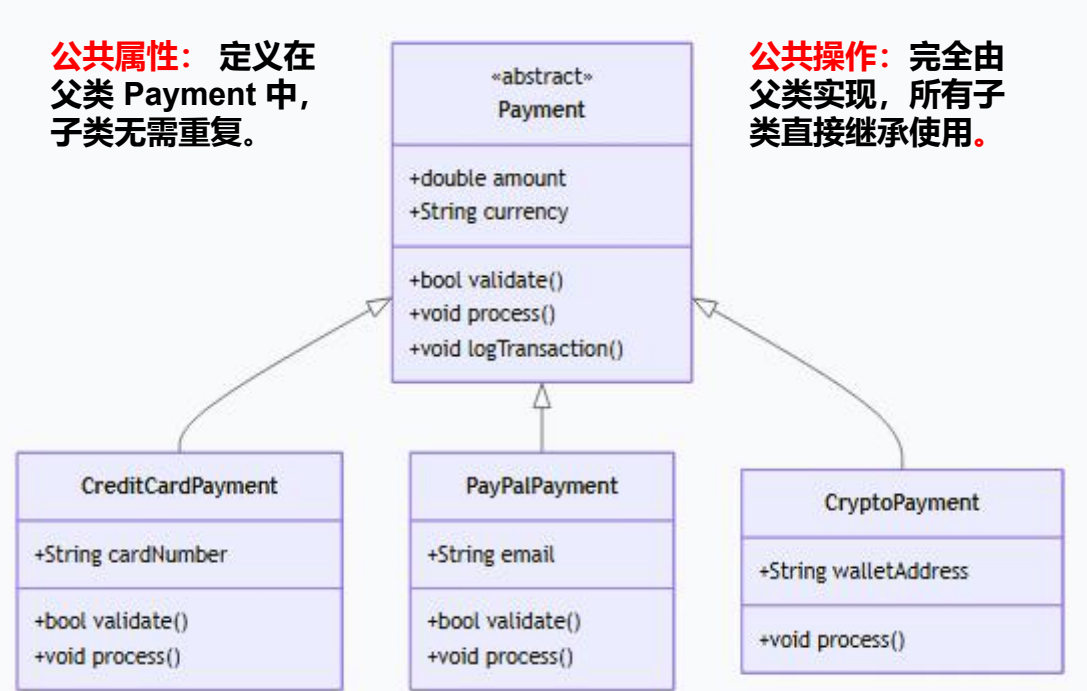
从服务导出操作：服务于状态绑定

义了读、

操作，是接受

“启动

力打对象的服务 更新账户。



公共属性：定义在父类 Payment 中，子类无需重复。

公共操作：完全由父类实现，所有子类直接继承使用。

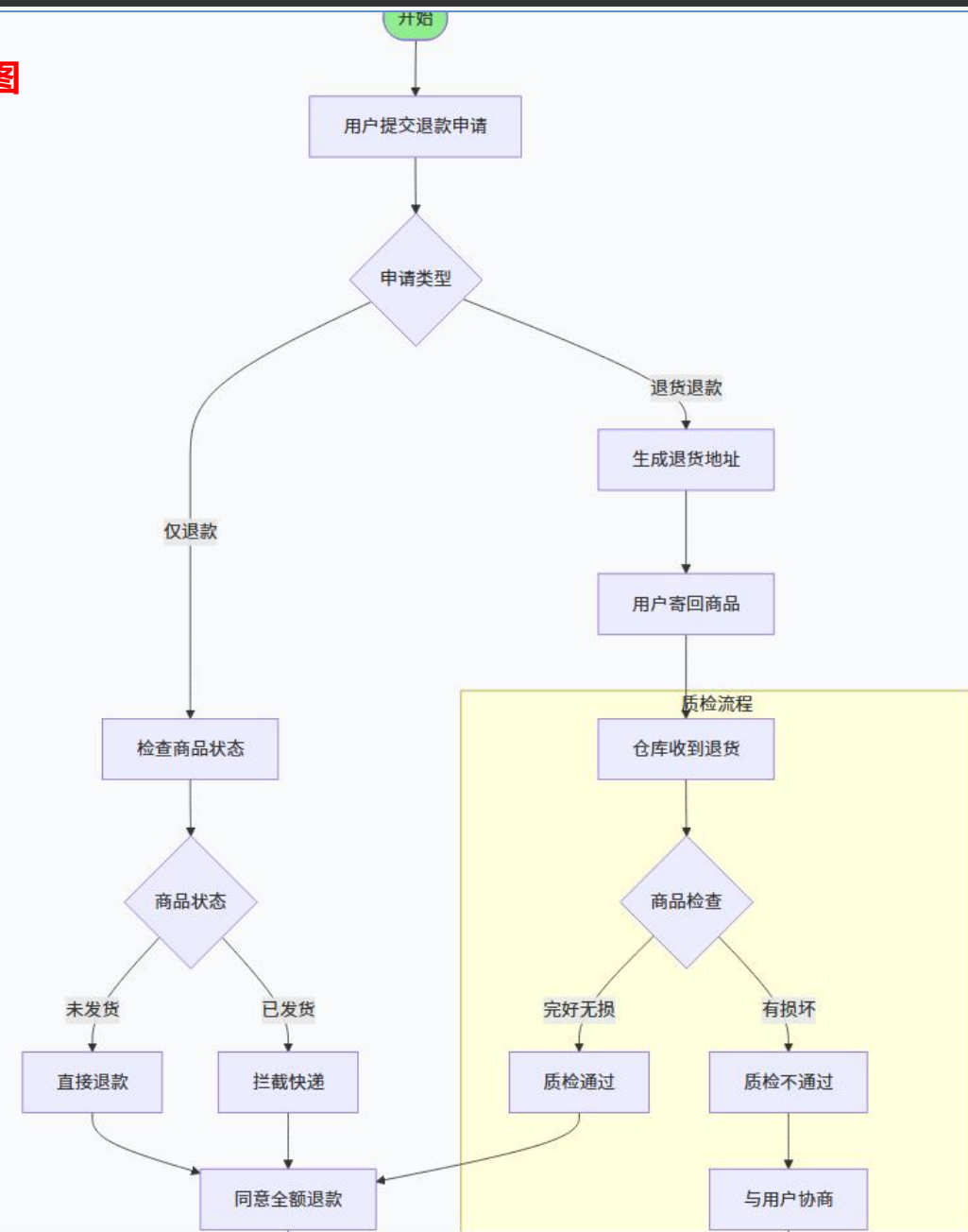
对应。应该仔细对照状

图上与之对应的处理框是“验

常识，就尽量抽取出相似准确地定义各个服务。



活动图



 **THANKS** 